

Промежуточный отчёт по лабораторной работе №4 по предмету “Технологии управления данными”

Текст задачи:

Лабораторная работа №4. Создания процедуры наполнения Хранилища данных.

Цель работы: создать хранимую процедуру, или скрипт, обеспечивающую автоматизированный перенос данных из Филиалов в Хранилище. Процедура должна обеспечить добавление вновь поступившей информации из Филиала Запад, и Филиала Восток в центральное хранилище данных. Процедура должна исключить дублирование информации. Разработчик должен продумать механизм обеспечивающий минимализацию нагрузки на продуктивные сервера филиалов.

В процедуре можно использовать операции множественной вставки.

Решение:

`last_added_date` - время последней вставки строки из данного филиала с данным первичным ключом в этом филиале в хранилище

`store_modified_date` - время изменения/добавления строки в филиал

Условие добавления в хранилище:

`last_added_date < store_modified_date` - значит строка в хранилище либо отсутствует, либо не актуальна

Если `last_added_date >= store_modified_date`, то строка в хранилище актуальна и ничего добавлять не нужно

Если `last_added_date` не существует (is None), то строки в хранилище не существует и её следует добавить в хранилище

Это исключает дублирование информации, сохраняя актуальность данных в хранилище данных

При этом ни одной операции удаления не выполняется

Также, в SQL-скрипт построения схемы хранилища данных было добавлено `GENERATED ALWAYS AS IDENTITY` для автоинкремента первичных ключей.

Код скрипта (на Python):

```
import psycopg2
```

```
db_config_west = {
    "dbname": "west_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5446"
}
```

```
db_config_east = {
    "dbname": "east_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5447"
}
```

```
db_config_warehouse = {
    "dbname": "warehouse_db",
```



```

warehouse_added_dates = sorted(cur_warehouse.fetchall(),
key=lambda x: x[1], reverse=True)
try:
    last_added_date = warehouse_added_dates[0][1]
except IndexError:
    last_added_date = None
if last_added_date is None or last_added_date <
store_modified_date:
    cur_warehouse.execute("INSERT INTO Product
(ProductName, StoreProductID, StoreID) "
                           "VALUES (%s, %s, %s) "
                           "RETURNING ProductID",
                           (name, ID, storeID))
    product_map[(int(ID), storeID)] =
(cur_warehouse.fetchone()[0], price)
else:
    product_map[(int(ID), storeID)] =
(warehouse_added_dates[0][0], price)
cur.execute("SELECT CategoryID, CategoryName, ModifiedDate "
            "FROM Category")
categories = cur.fetchall()
for category in categories:
    ID = str(category[0])
    name = str(category[1])
    store_modified_date = category[2]
    cur_warehouse.execute("SELECT CategoryID, AddedDate "
                           "FROM Category "
                           "WHERE StoreCategoryID = %s AND
StoreID = %s",
                           (ID, storeID))
    warehouse_added_dates = sorted(cur_warehouse.fetchall(),
key=lambda x: x[1], reverse=True)
try:
    last_added_date = warehouse_added_dates[0][1]
except IndexError:
    last_added_date = None
if last_added_date is None or last_added_date <
store_modified_date:
    cur_warehouse.execute("INSERT INTO Category
(CategoryName, StoreCategoryID, StoreID) "
                           "VALUES (%s, %s, %s) "
                           "RETURNING CategoryID",
                           (name, ID, storeID))
    category_map[(int(ID), storeID)] =
cur_warehouse.fetchone()[0]
else:
    category_map[(int(ID), storeID)] =
warehouse_added_dates[0][0]

```

```

cur.execute("SELECT CategoryID, ProductID "
            "FROM ProductCategory")
product_categories = cur.fetchall()
for store_cID, store_pID in product_categories:
    cID = category_map[(store_cID, storeID)]
    pID = product_map[(store_pID, storeID)][0]
    cur_warehouse.execute(
        "INSERT INTO ProductCategory (CategoryID, ProductID)
"
        "VALUES (%s, %s)",
        (cID, pID))
cur.execute("SELECT CustomerID, CustomerFirstName,
CustomerLastName, ModifiedDate "
            "FROM Customer")
customers = cur.fetchall()
for customer in customers:
    cID = customer[0]
    first_name = customer[1]
    last_name = customer[2]
    cur.execute("SELECT ReceiptID, TotalPrice, CompletedDate
"
                "FROM Receipt "
                "WHERE CustomerID = %s",
                (cID,))
    store_modified_date = customer[3]
    cur_warehouse.execute("SELECT CustomerID, AddedDate "
                           "FROM Customer "
                           "WHERE StoreCustomerID = %s AND
StoreID = %s",
                           (cID, storeID))
    warehouse_added_dates = sorted(cur_warehouse.fetchall(),
key=lambda x: x[1], reverse=True)
    try:
        last_added_date = warehouse_added_dates[0][1]
    except IndexError:
        last_added_date = None
    if last_added_date is None or last_added_date <
store_modified_date:
        cur_warehouse.execute(
            "INSERT INTO Customer (StoreCustomerID, StoreID,
CustomerFirstName, CustomerLastName) "
            "VALUES (%s, %s, %s, %s) RETURNING CustomerID",
            (cID, storeID, first_name, last_name))
        customerID = cur_warehouse.fetchone()[0]
    else:
        customerID = warehouse_added_dates[0][0]
    receipts = cur.fetchall()
    for receipt in receipts:

```

```

rID = receipt[0]
total_price = receipt[1]
completed_date = receipt[2]
cur.execute("SELECT ProductID, ActualPrice, Amount "
            "FROM ReceiptItem "
            "WHERE ReceiptID = %s",
            (rID,))
cur_warehouse.execute("SELECT AddedDate "
                      "FROM Receipt "
                      "WHERE StoreReceiptID = %s AND
StoreID = %s",
                      (rID, storeID))
warehouse_added_dates =
sorted(cur_warehouse.fetchall(), key=lambda x: x[0], reverse=True)
try:
    last_added_date = warehouse_added_dates[0][0]
except IndexError:
    last_added_date = None
if last_added_date is None or last_added_date <
completed_date:
    cur_warehouse.execute(
        "INSERT INTO Receipt (StoreReceiptID,
StoreID, TotalPrice, CompletedDate) "
        "VALUES (%s, %s, %s, %s) RETURNING
ReceiptID",
        (rID, storeID, total_price, completed_date))
    receiptID = cur_warehouse.fetchone()[0]
    items = cur.fetchall()
    for item in items:
        product = item[0]
        productID, listed_price =
product_map[(product, storeID)]
        price = item[1]
        amount = item[2]
        year = completed_date.year
        month = completed_date.month
        day = completed_date.day
        weekday = completed_date.weekday()
        weekdayname = weekdays[weekday]
        monthname = months[month - 1]
        date = f'{year:04}-{month:02}-{day:02}'
        dateID = f'{year:04}{month:02}{day:02}'
        if date not in dates:
            dates.add(date)
            cur_warehouse.execute(
                "INSERT INTO Date (DateID, Day,
Month, Year, Weekday, MonthName, WeekdayName)"

```

```

        " VALUES (%s, %s, %s, %s, %s, %s,
%s)",
        (dateID, day, month, year, weekday,
monthname, weekdayname))
        cur_warehouse.execute(
            "INSERT INTO SoldItem (ReceiptID,
CustomerID, ProductID, StoreID, DateID, ListedPrice, ActualPrice, Amount)"
            " VALUES (%s, %s, %s, %s, %s, %s, %s,
%s)",
            (receiptID, customerID, productID,
storeID, dateID, listed_price, price, amount))
        conn_warehouse.commit()
except (Exception, psycopg2.DatabaseError) as error:
    if conn_warehouse:
        conn_warehouse.rollback()
    print("Error with PostgreSQL:", error)
finally:
    if conn_warehouse:
        conn_warehouse.close()
    if conn_east:
        conn_east.close()
    if conn_west:
        conn_west.close()
print("All done!")

```